

* Foundation of Web Communication:-

To understand how modern frameworks work, we must first look at how users interact with a website:

→ Client - Server Architecture:-

The client → Any entity that requests information (e.g. a web browser, mobile app).

The server → A computer hosting the application that receives the requests, processes it, and sends back a response.

The workflow → The user hits a URL, the server processes the logic (like authentication or data retrieval), and returns a response that browser displays.

→ HTTP/HTTPS Protocol:-

HTTP (Hypertext Transfer Protocol) → Acts as the "rule book" for communication. It defines the language used between the client and server so they can understand each other.

HTTPS → The 'S' stands for Secured. It ensures that data transferred between client and server is encrypted, protecting it from being intercepted.

→ Request & Response Structure :-

Request Components →

1. Method Name = The action requested (e.g. GET to fetch data, POST to store data).
2. URL/Endpoint = The specific path on the server (e.g. /courses).
3. Headers = Metadata like the format of data expected (Application/JSON) or authentication details.
4. Body = The actual content sent to the server (common in POST requests).

Response Components →

1. Status Code = Indicates the outcome (e.g. 200 OK for success, 404 Not Found).
2. Headers = Metadata regarding the response.
3. Body = The actual data (often in JSON format) sent back to user.

* Moving from Core Java to Web Development :-

Transitioning from standard Java code to web applications involves solving unique networking challenges.

→ The Networking Problem :-

Core Java limitations → While Java's `java.net` package allows developers to listen for network requests using `Socket` or `ServerSocket` on specific ports (like 8080), the code doesn't natively understand HTTP protocols.

Date.....

- Manual Effort → Without a framework, developers must manually:
- Read input streams (bytes).
 - Parse HTTP requests (decipher GET, headers, body, etc.).
 - Map every URL endpoints to a specific Java method.
 - Manually build an HTTP response and handle multi-threading for concurrent users.

→ Solution: Servlets & Servlet Containers :-

Servlet Containers (e.g. Tomcat) = A powerful tool that automates the "boiler-plate" work. It handles ports management, request parsing, and threading.

Servlets = A Java class that runs inside the containers. The containers routes incoming requests to the appropriate Servlet, allowing the developer to focus purely on business logic rather than network plumbing.

→ The Birth of Spring :-

- As application scale, Servlet-based code can become "tightly coupled" and hard to manage.
- Spring Framework was introduced to create a more modular, "loosely coupled" architecture, making enterprise applications easier to maintain and scale.

* The Spring Ecosystem :-

Moving beyond traditional Servlets, Spring Framework provides a comprehensive ecosystem designed for scalability and modularity.

→ Core Concepts:

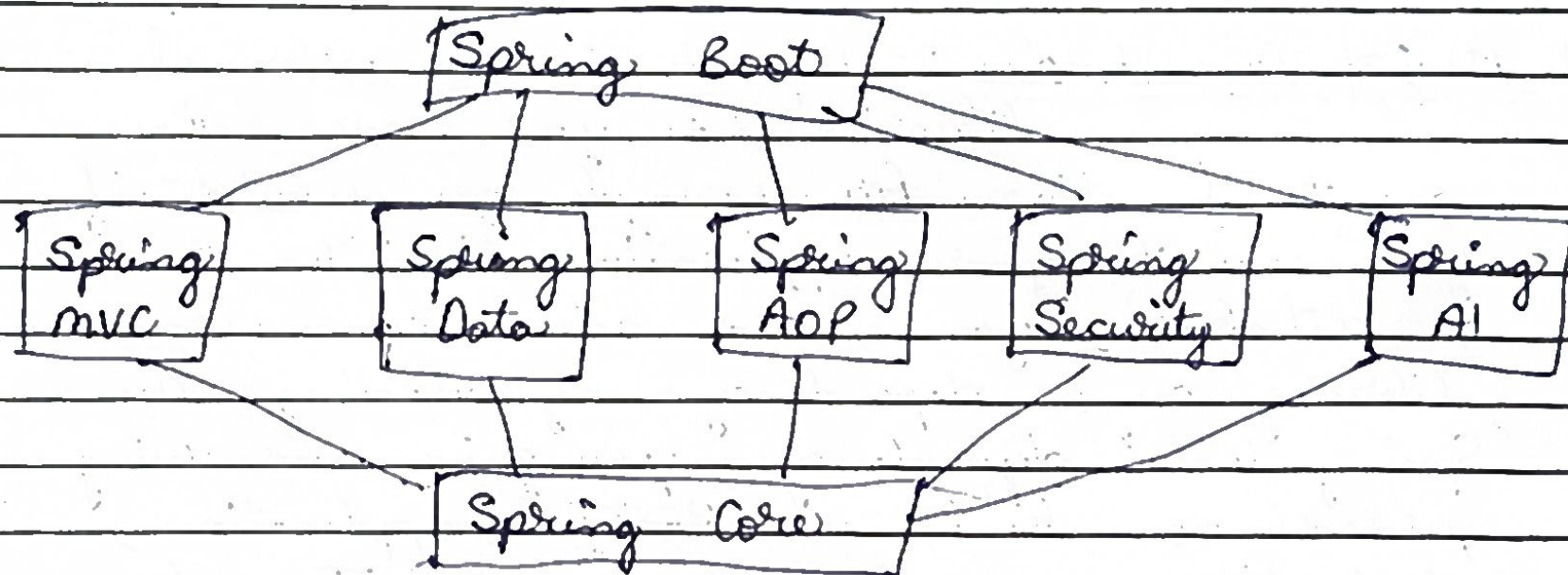
- Loose Coupling → Spring replaces rigid, tightly linked objects with flexible designs using principles like Dependency Injection and Inversion of Control.
- Ecosystem → It is not just one framework but a collection of modules that handle everything from security to data management.

→ Key Spring Modules:

- Spring Core → The foundation (IoC and Dependency Injection) that everything else is built upon.
- Spring MVC → Simplifies building web applications, abstracting away much of manual work required by standard Servlets.
- Spring Data → Connects the application to databases. It evolved from manual JDBC (Java Database Connectivity) to modern approaches like JPA (Java Persistent API) and Hibernate.
- Spring Security → Handles authentication and authorization to keep applications safe.

Date.....

- Spring Boot → The automation layer built on top of Spring framework. It is an "opinionated" system that handles complex configurations automatically, allowing developers to start coding business logic immediately.



* IP Address :- Every device on a network has a unique IP Address. When you run an application locally, you use the loopback address [127.0.0.1] commonly referred to as localhost.

* Port Numbers :- Since a single computer runs multiple applications simultaneously (like a browser, Spotify, or your server), Port Numbers act as gates. They ensure that incoming data reaches the correct application. Spring Boot applications typically run on port 8080 by default.

Date.....

For internal communications, standard ports are used: HTTP requests go to port 80, and HTTPS requests go to port 443. If an application is running on a different port (like 8080), a Reverse Proxy is often used to map the standard traffic to your application's specific port.

* Project Dependencies :- Dependencies are external libraries that your code relies on to function. For example, you would if you want your application to interact with a database, you would add a connector library as a dependency. By selecting the 'Spring Web' dependency, you gain everything needed to build a RESTful application, including an embedded Apache Tomcat server.

Let's Create Our First Spring App :-

Here, we downloaded a sample file from "Spring Initializer Site".

Then, we make a new file named "HelloController" and makes a API route so that when in browser we type : ("localhost:8080/hello") we see "Hello World". Same for /bye API.

By default, it runs on "8080" port but we can change that by modifying [application.properties] file.

HelloController.java

Date.....

Sagar 02K4

```
@RestController
public class HelloController {

    @GetMapping("hello")
    public String hello() {
        return "Hello World";
    }

    @GetMapping("bye")
    public String bye() {
        return "Bye";
    }
}
```

The Controller → To handle web requests, you create a Controller class. By adding the `@RestController` annotation, you tell Spring that this class will handle incoming web requests.

The Endpoints → You define your first API endpoints using the `@GetMapping` annotation. For example, by using `@GetMapping("/hello")`, you map a request to `localhost:8080/hello` to a specific Java method that returns the string "Hello World".